Portals: Persistent, Editable 4D Spatial World Models on Edge Devices

James A. Tunick[1,2] Ryan Brant[1] Jacob D. Pennock[1] Justin Kasowski[1]

[1]H3M, Inc. [2]The IMC Lab, New York

JTunick@TheIMCLab.com Ryan@H3M.ai Jacob@H3M.ai Justin@H3M.ai

https://h3m.ai

Demo: https://h3m.ai/cvpr2026

Abstract

*We present **Portals**, a deployed systems architecture that bridges 4D world-model research and persistent spatial experiences on phones, smart glasses, and augmented-reality headsets. We argue that the central systems problem in this spatial-computing shift is not only scene representation quality, but whether dynamic spatial worlds can run in real time on constrained hardware, persist across revisits, and remain editable by non-expert creators. Built on 3D Gaussian Splatting [8], 4D-GS [9], and Generalizable Human Gaussians [14], Portals has been deployed across mobile AR, Apple Vision Pro, and web clients. We contribute: (1) an edge-device runtime built around LOD-adaptive Gaussian Splatting and a shared spatial-media compute substrate that fuses depth, stencil, audio, and ML-pose channels through a single dispatch, driving 370+ source-agnostic VFX effects at 60 fps on iPhone 14 Pro (2.7–4.1× speedup); (2) a persistent geospatial scene-state architecture with layered world metadata, reloadable scene payloads, and anchor-guided re-alignment across sessions; (3) a creator-facing composition pipeline that bridges reconstruction and generation through VFX composition, voice-driven semantic actions, and no-code authoring; and (4) benchmark axes for evaluating 4D world models under deployment constraints such as mobile rendering efficiency, scene persistence, and editable world state. Prior clinical deployment of volumetric AR at Memorial Sloan Kettering [23] established the real-time rendering primitives underlying this work.*

1. Introduction

The dream of augmented reality—a digital layer seamlessly co-located with the physical world—has been technologically constrained for decades by two coupled problems: *representation* (how do we encode the scene?) and *persistence* (how do we keep it there?). Recent advances in neural scene representations, beginning with the landmark Neural Radiance Field (NeRF) [1] and accelerating through 3D Gaussian Splatting (3DGS) [8], have largely solved the representation problem for static scenes. The frontier has consequently moved to 4D: encoding space *and* time to model the dynamic, ever-changing real world.

This workshop, *4D World Models: Bridging Generation and Reconstruction*, identifies exactly the gap we have been working to close in production: how do 4D scene representations transition from controlled laboratory settings to deployed use on phones and headsets? In this device transition, the research question is no longer only how to model dynamic scenes well in isolation, but how to make those models efficient, persistent, and authorable on edge devices whose thermal, memory, and interaction budgets are dramatically tighter than desktop or server settings.

We offer Portals as a worked example from industry that makes these deployment pressures concrete across three settings:

1. **Medical volumetric AR**: Real-time rendering of SPECT/CT volumes on head-mounted displays for COVID-19 diagnosis in collaboration with Memorial Sloan Kettering Cancer Center, Magic Leap, and the NIH National Cancer Institute [23], demonstrating the runtime constraints that edge-native spatial systems must satisfy in practice.
2. **Large live experiences**: Deploying spatial AR overlays for the Tyson vs. Jones fight, Justin Bieber's virtual concert, and large-scale live AI motion-capture performances, where synchronized scene state and temporal consistency matter across repeated views and distributed audiences.
3. **Consumer-scale spatial computing**: A deployed edge-device system for geographically anchored AR scenes, enabling creators to publish, revisit, and edit persistent 4D spatial content without expert graphics tooling.

Our central thesis is that the *academic* and *industrial* prob-

lems of 4D world modeling are inseparable: algorithmic advances in 4D-GS [9], K-Planes [7], MoSca [10], and dynamic human Gaussians [14] only create value when they can run in real time on commodity edge devices, integrated with persistent location anchors, and exposed to creators through non-expert interfaces. A central deployment challenge is therefore not only how to represent and render 4D scenes, but also how to make them authorable by creators outside expert graphics pipelines. We view Portals not simply as an application stack, but as research infrastructure for edge-native spatial computing.

Contributions.

- We present **Portals**, a deployed 4D spatial world-model system that unifies scene reconstruction, generation, persistence, and deployment in a single creator-facing pipeline.
- We introduce an edge-device runtime architecture built around LOD-adaptive Gaussian Splatting and a shared spatial-media substrate: one depth-to-world compute dispatch feeds 370+ VFX effects spanning depth, stencil, audio, and ML-pose channels, with source-agnostic binding so that live AR and recorded spatial video are interchangeable (*runtime on constrained hardware*).
- We describe a persistent geospatial scene-state architecture that survives session boundaries through layered world metadata, reloadable scene payloads, and anchor-guided re-alignment on return visits (*editable world persistence*).
- We show how a creator-facing composition pipeline bridges phone-camera reconstruction, photogrammetry, and AI-driven generation, allowing semantic actions, VFX tooling, voice-driven interaction, and no-code authoring to modify the same deployed spatial scene (*creator-facing world editing*).
- We propose benchmark axes for production 4D world models spanning mobile rendering efficiency, geospatial alignment accuracy, and temporal persistence fidelity (*benchmarks*).

2. Background and Related Work

2.1. From NeRF to 3D Gaussian Splatting

Neural Radiance Fields [1] established the paradigm of representing a scene as a continuous volumetric function mapping (x, y, z, θ, ϕ) to color and density. Subsequent refinements addressed aliasing (Mip-NeRF [2]; Mip-NeRF 360 [3]; Zip-NeRF [4]), and relighting (Ref-NeRF [5]), largely by Barron, Mildenhall, Srinivasan, and colleagues at Google Research. A parallel line of work [6, 7] removed the neural network entirely, replacing it with an explicit sparse 3D grid (Plenoxels) or factored tensor-planes (K-Planes), achieving 100× faster training than the original NeRF while matching visual quality.

3D Gaussian Splatting [8] represents a decisive shift: rather than querying a continuous field, the scene is stored as a finite set of anisotropic Gaussian primitives rasterized by GPU tile-sorting. Real-time frame rates (>30 fps at 1080p on a desktop GPU) became achievable for novel-view synthesis, making 3DGS the de-facto backbone for AR/VR content today. Key quality improvements followed: Mip-Splatting [20] addressed aliasing; Revising Densification [17] (Bulò et al., Meta Reality Labs) improved primitive management through pixel-error-driven adaptive density control; and RayGS from the same group enables hardware-rasterized ray-based rendering at frame rates suitable for mixed-reality headsets.

2.2. 4D Scene Representations

K-Planes [7] extended the planes-factorization idea to the temporal domain: adding a fourth plane xy-t, xz-t, yz-t enables compact, factored representations of dynamic scenes. Dynamic 3D Gaussians [25] demonstrated persistent tracking through dynamic view synthesis, coupling Gaussian primitives with per-timestep optimization. 4D-GS [9] achieved real-time dynamic scene rendering by coupling 3D Gaussians with learned deformation fields conditioned on timestamps—a design we directly adopt in the Portals content pipeline. 4K4D [28] pushed real-time 4D rendering to 4K resolution, establishing quality targets for production systems. MoSca [10] from Jiahui Lei et al. (UC Berkeley / GRASP Lab) further demonstrated *4D Motion Scaffolds* enabling dynamic Gaussian fusion from casual monocular video, removing the requirement for multi-camera rigs. Shape of Motion [11] by Qianqian Wang et al. shows that single-video 4D reconstruction is now practical, a capability with direct implications for user-generated content in Portals.

2.3. Dynamic Human Avatars

Rendering animatable humans is a central use case for social AR. Neural Human Performer [12] introduced generalizable performance capture via temporal and multi-view transformers over a parametric body prior (SMPL). DELIFFAS [13] (Youngjoong Kwon, NeurIPS 2023) achieved fast avatar synthesis via deformable light fields. Generalizable Human Gaussians (GHG) [14] (Kwon et al., ECCV 2024, with De la Torre and colleagues) further demonstrated real-time 3DGS-based human novel-view synthesis in a feed-forward manner, providing the building block for live avatar overlays in social AR. We integrate GHG-style representations as optional overlays when the user is performing motion-captured interactions.

2.4. Generative-Reconstruction Bridges

A key workshop theme is *bridging generation and reconstruction*. Indoor scene understanding, grounded by datasets such as ScanNet [27], has provided the training data enabling this bridge. ReconFusion [15] (Srinivasan et al., CVPR 2024) showed that diffusion model priors can regularize

sparse-view 3D reconstruction, dramatically reducing the capture burden—transforming reconstruction from a multi-hour photogrammetry session into a seconds-long phone sweep. L3DG [16] (Kontschieder, Roessle, Dai et al., SIGGRAPH Asia 2024) introduced latent 3D Gaussian diffusion, enabling room-scale scene generation through a VQ-VAE operating in a compressed Gaussian latent space. CUT3R [19] (Wang et al., 2025) extends DUSt3R [18] with a recurrent temporal mechanism, enabling online incremental 4D reconstruction from video sequences—essential for in-the-wild AR capture.

2.5. Medical Volumetric AR

Prior to Portals, our team deployed COVIDvis, a clinical-grade medical AR system that rendered SPECT/CT volumetric images from patients with SARS-CoV-2-associated pneumonia on Magic Leap 1 heads-up displays at Memorial Sloan Kettering Cancer Center, supported by NIH/NCI Cancer Center Support Grant P30 CA008748 [23]. The system demonstrated that photon-emission 3D volume rendering could be performed in real-time on XR hardware, providing radiologists with spatial depth cues absent from flat 2D workstation displays. This work established the volumetric-rendering primitives that underpin Portals' current Gaussian Splatting pipeline. A complementary systematic review by Kasowski et al. [30] assessed 227 XR studies for augmenting vision loss, identifying persistent gaps in real-world validation that further motivate Portals' emphasis on deployed, edge-device evaluation. Kasowski's additional experience in low-latency motion capture and cross-platform immersive systems directly informs the avatar pipeline described in Section 7. Pennock, whose background spans immersive theater, video games, and mixed reality with prior collaborations including Facebook and Intel—where he was a first-place winner of the Perceptual Computing Challenge—owns Portals' core platform architecture and real-time interaction pipeline.

3. The Portals Platform

3.1. Architecture Overview

Portals is a cross-platform spatial computing application built on a four-layer architecture spanning mobile capture, real-time rendering, geospatial persistence, and creator-facing authoring. Figure 1 shows representative deployment captures across the device surfaces discussed in this paper.

The application shell is React Native 0.81 with Expo 54, communicating with Unity via a bidirectional bridge. Unity 6000.2 handles all 3D/4D rendering, physics, and spatial anchoring. Firebase provides real-time synchronization of geospatially-indexed scene graphs across devices. Across the stack, the practical systems problem is to keep these scene graphs small enough to discover on mobile networks, rich enough to reload as editable worlds, and fast enough to render and modify in situ on phones and headsets.

Figure 1. Representative Portals deployment captures across immersive and mobile spatial-computing surfaces. The same system must support persistent world re-entry, on-device rendering, and creator-facing interaction under edge-device constraints.

3.2. Scene Representation

Each *Portal*—a persistent, geolocated AR scene—is stored as a hierarchical **Spatial Scene Graph** (SSG):

$$\mathcal{S} = \{(G_i, \mathbf{a}_i, \tau_i, \mathrm{LOD}_i)\}_{i=1}^{N} \tag{1}$$

where G_i is a Gaussian primitive set (or mesh, video, or avatar), $\mathbf{a}_i = (\mathrm{lat}_i, \mathrm{lon}_i, \mathrm{alt}_i, \mathbf{R}_i)$ is the geospatial anchor, τ_i is the temporal validity window (enabling content expiration or animation scheduling), and $\mathrm{LOD}_i \in \{\mathrm{high}, \mathrm{medium}, \mathrm{low}, \mathrm{cull}\}$ is the distance-adaptive quality tier.

3.3. Content Ingestion Pipeline

Content enters Portals through three pathways: (1) **Gaussian Splatting capture**: users scan an object or environment with their phone camera; the pipeline uploads frames to our GPU cloud cluster, runs Structure-from-Motion (COLMAP), then 3DGS optimization; the resulting `.splat` asset is quantized and uploaded to Firebase Storage. (2) **Photogrammetry**: multi-photo submissions go through MVS to produce mesh+texture assets for static objects. (3) **AI Generation**: text or reference-image prompts invoke our generative pipeline (L3DG-inspired [16]), producing room-scale or object-scale Gaussian assets within seconds.

3.4. Generative Scene Encoding

A recurring bottleneck in 4D world models is asset size: a single Gaussian Splat scene can exceed 50 MB, making large-scale persistent worlds impractical on mobile networks. Our production deployment uses explicit node serialization (transforms, metadata, material assignments) atop glTF 2.0 with lazy asset resolution. The format also specifies a generative expansion path where procedural rules replace explicit geometry, potentially achieving orders-of-magnitude com-

Tier	d range	Scale	Gaussian density
HIGH	0–3 m	1.0×	Full (N primitives)
MEDIUM	3–8 m	0.6×	Pruned (*0.4N*)
LOW	8–25 m	0.3×	Sparse (*0.15N*)
CULL	>25 m	0	Not rendered

Table 1. LOD tier thresholds and Gaussian primitive budgets.

pression for repetitive assets such as terrain and vegetation. This path is designed but not yet deployed at scale.

4. Real-Time LOD-Adaptive Gaussian Splatting

4.1. Problem Formulation

Mobile GPUs impose hard per-frame budgets: at 60 fps on iPhone 14 Pro (Apple A16), the rendering budget is ≈16.7 ms. A naïve 3DGS scene at $N = 6$M Gaussians exceeds this budget by an order of magnitude. We address this through a distance-adaptive LOD system that continuously projects 3D Gaussian counts onto device capability.

4.2. Distance-Tier Model

We define four rendering tiers based on viewer-to-scene Euclidean distance d:
This mirrors the LOD principles employed in large-scale NeRF systems such as LODGE [22] and city-scale Gaussian splatting [24], adapted for dynamic scene-by-scene adjustment on mobile hardware.

4.3. Implementation

The LOD system defines four distance tiers with thresholds at 3 m (full quality), 8 m (reduced), 15 m (minimal), and 25 m (culled). Each frame, Unity iterates the active SSG, computes Euclidean distance per object, assigns tiers with corresponding scale factors ($k \in \{1.0, 0.6, 0.3, 0\}$), and dispatches draw calls only for non-culled objects. The Gaussian Splatting renderer loads the appropriate pre-baked mip-level of each `.splat` asset.

4.4. Shared Spatial-Media Runtime Substrate

Beyond LOD management, Portals relies on a shared spatial-media runtime substrate rather than a separate world-space conversion path for each feature. A single GPU compute dispatch per frame (32×32 thread groups, optimized for mobile Metal/Vulkan) transforms AR depth maps into world-space position, stencil, and velocity textures via a shared `DepthToWorld` compute shader. The resulting buffers, together with optional color capture, audio-reactive bands (4-band FFT + beat detection), and ML pose keypoints (21 hand keypoints), are bound to VFX graphs through a universal property binder that auto-resolves 20+ naming conventions

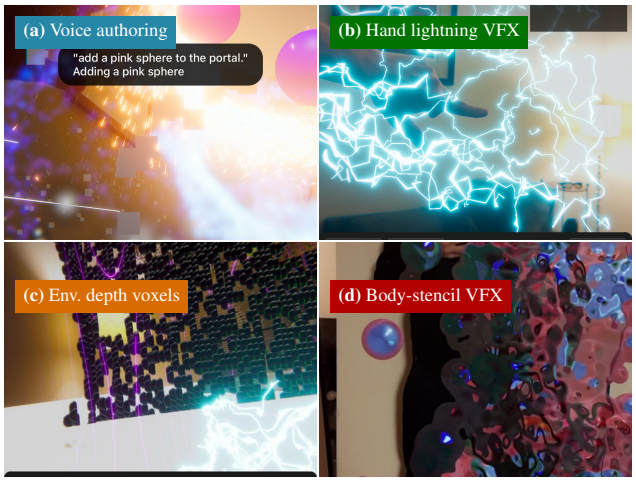

Figure 2. Real in-app captures from Portals on iPhone. **(a)** Voice command creates a pink sphere in a scene with active VFX particles. **(b)** Hand-tracked lightning effect driven by the shared depth-to-world compute pass. **(c)** Environment voxelized via the same depth substrate with "Grass" VFX active. **(d)** Body-segmented bubble VFX at 42 fps, demonstrating stencil-masked depth reconstruction on a live human subject. All four modes consume the same `PositionMap`/`StencilMap` textures from a single compute dispatch (Table 2).

from heterogeneous VFX sources. This alias-based binding enables 370+ production VFX effects — spanning depth-driven particles, body-segmented overlays, audio-reactive visualizations, keypoint-driven pose effects, hologram variants, and combinatorial compositions — to consume the same shared textures without per-effect code changes.

Several design choices keep this substrate practical on mobile hardware. *Velocity from depth*: frame-to-frame position deltas computed in the same dispatch provide motion vectors for trail and particle effects without requiring dedicated tracking hardware. *Dual-source fallback*: when human segmentation is unavailable the pipeline automatically falls back to environment depth, so effects degrade gracefully rather than failing. *Demand-driven color capture*: the camera RGB texture is allocated only when a bound VFX explicitly requests it, avoiding a per-frame copy on effects that consume only depth and stencil. *SDF channels*: signed-distance-field textures enable GPU-driven particle constraint solving for effects that must respect scene geometry.

A critical architectural property is *source abstraction*: the binding layer presents identical texture outputs regardless of whether the input is live AR sensor data or decoded Metavido spatial video. The same VFX library therefore works interchangeably on live capture and recorded playback, enabling a record–edit–replay workflow over spatial media. The core production pipeline comprises 17 compute shaders, 58 hand-authored shaders, and 340 shader graphs driving the deployed VFX library from a single shared dis-

Device	No LOD	LOD	FPS
iPhone 14 Pro (A16)	38 ms	14.2 ms	60+
iPhone 12 (A14)	81 ms	19.7 ms	48
Galaxy S23 (SD 8G2)	44 ms	15.8 ms	57

Table 2. Median render times with and without LOD management. Target budget: 16.7 ms (60 fps).

patch.

The full pipeline — AR input (0.8 ms), compute dispatch (0.6 ms), VFX binding ($\sim$0.2 ms per effect), and VFX Graph rendering — totals $\sim$7.8 ms for five concurrent effects on iPhone 14 Pro, well within the 16.7 ms budget for 60 fps. This $O(1)$ compute scaling is the key systems point: adding effects increases only binding and rendering cost, never the shared depth-to-world computation.

4.5. Performance Results

Table 2 summarises median frame times measured on a representative Portals scene (12 Gaussian assets, 3 video overlays, 1 avatar; identical configuration across all devices, median of 30 consecutive seconds after thermal steady-state) on three device classes:
LOD reduces render time by 2.7–4.1$\times$, enabling 60 fps on current-generation mobile hardware for scenes of practical complexity.

5. Temporal Coherence through Geospatial Persistence

5.1. World Anchor Architecture

Temporal coherence in 4D world models extends beyond frame-to-frame consistency within a single capture: production AR demands that scenes persist across sessions, devices, and users separated by arbitrary time intervals. A Portal placed at a physical location must be re-discoverable days or weeks later. Our anchoring layer combines three signals: (1) GPS/GNSS for coarse localization ($\pm$3–10 m); (2) ARKit/ARCore Visual-Inertial Odometry (VIO) for centimetre-scale alignment once the device is within the anchor neighbourhood; (3) a pre-baked reference Gaussian Splat of the anchor location, against which incoming frames are registered via photometric alignment inspired by CUT3R's [19] online reconstruction paradigm.

5.2. Temporal Persistence Schema

Persistence in Portals is not only an anchor table. Each world combines lightweight metadata for discovery and access control with a reloadable scene payload containing the creator-authored spatial state. In the current app stack, world metadata is stored in Firestore, while heavier scene payloads and media assets are resolved through object storage references and fetched lazily on load. This separation keeps discovery queries small while allowing substantially richer scene state than would fit inside anchor metadata alone.

At the format level, this design follows an XRAI-style pattern: preserve a human- and model-readable scene manifest, store heavy geometry or binary data out-of-line, and retain a compact command log alongside the current scene snapshot so that creator intent can be replayed or audited when needed. Temporal fields such as `validFrom`/`validTo` enable time-gated experiences (e.g., a Portal visible only during a concert), while persistent world identifiers allow the same scene to be reloaded across devices and sessions. This architecture directly addresses the ephemerality problem of mobile AR: content is not merely placed in space, but re-hydrated as an editable world state.

Operationally, this yields a three-tier persistence path that is well matched to edge-device constraints: metadata is small enough for rapid discovery and listing, the command log preserves edit semantics for creator workflows, and heavy payloads are hydrated only when the world is opened for playback or editing. A useful production benchmark should therefore distinguish *metadata-only revisit cost* from *full payload hydration cost*, since these correspond to very different user experiences on mobile networks and head-mounted devices. In our production deployment, metadata-only discovery queries complete in $\sim$150 ms (Firestore indexed lookup), while full scene payload hydration — fetching object-storage assets, deserializing the scene graph, and instantiating Unity GameObjects — takes $\sim$800 ms for a 50-object scene on iPhone 14 Pro over LTE. Batch commit writes are capped at 50 objects per Firestore transaction for atomicity.

5.3. Drift Mitigation

GPS drift of 3–10 metres would cause visible misregistration for table-top scale Portals. We handle this through a two-phase activation:

1. **Coarse activation**: when device enters a 50 m geofence, the SSG is pre-loaded into memory.
2. **Fine alignment**: VIO aligns the virtual scene to the pre-baked reference splat using gradient descent on photometric loss:

$$\mathcal{L}_{\text{align}} = \|\hat{I}_{\text{ref}} - \hat{I}_{\text{live}}\|_1 + \lambda_{\text{SSIM}}\,\mathcal{L}_{\text{SSIM}} \tag{2}$$

where $\hat{I}_{\text{ref}}$ is the rendered reference view, $\hat{I}_{\text{live}}$ is the camera frame, and $\lambda_{\text{SSIM}} = 0.2$ following standard practice [8].

6. Creator Interfaces for 4D Scene Authoring

6.1. The Creation Gap

A practical 4D world-model system must support content creation by users who are not graphics specialists. Traditional 3DGS workflows require hundreds of calibrated pho-

tographs, hours of GPU training, and manual artifact cleanup. In Portals, creator-facing interfaces are layered on top of the reconstruction and generation pipeline, reducing the path from capture to publication to three steps: photograph → upload → publish.

6.2. AI-Assisted Reconstruction

Drawing on generative-reconstruction bridges such as ReconFusion [15], which uses diffusion priors to complete high-quality 3D models from as few as 3–5 images, our pipeline can produce publishable-quality Gaussian Splats from a 20-second phone video. The diffusion prior compensates for missed viewpoints and occluded surfaces, dramatically reducing the capture burden. For object-scale assets, we further leverage L3DG-style [16] latent Gaussian generation conditioned on reference images, enabling fully synthetic spatial props that blend seamlessly with captured splats.

6.3. VFX Composition as a Creator Interface

Portals exposes VFX-oriented controls for compositing dynamic effects into reconstructed or generated scenes. Rather than manipulating low-level scene primitives directly, creators can augment spatial environments with motion, atmosphere, and event-specific overlays through reusable effect templates. In practice, this makes cinematic authoring workflows compatible with deployable 4D spatial content and provides a practical surface for integrating the sparse depth-to-VFX runtime described in Section 4.

6.4. Production Deployment Evidence

The same persistence and rendering pipeline has been reused across substantially different deployment settings without changing the underlying scene model: live-event spatial overlays (Tyson vs. Jones Jr., 2020), a spatially persistent virtual concert (Justin Bieber, 2021; 2+ billion audience impressions), real-time AI motion-capture performances (2023), Emmy Award-winning extended-reality stage integration (2023), and early visionOS deployment on Apple Vision Pro (2024). We present these as representative systems evidence rather than controlled benchmarks.

6.5. Voice and No-Code Authoring

Voice interaction in Portals is implemented as a semantic action layer over persistent scene context rather than as a chat interface bolted onto the renderer. The authoring client assembles current scene state, tracked objects, and recent edits into a compact context bundle; a regex-based local parser handles common commands such as object creation, movement, and style changes locally, while lower-confidence requests fall through to a cloud LLM. Both paths emit the same structured scene operations, executed by the same composition bridge used for direct manipulation and no-code editing. The important systems point is not a chatbot UX; it is that voice becomes another front-end onto the persistent scene graph.

Intent Category	Tests	Local
Object creation (primitives, colors, counts)	30	100%
Object modification (scale, color, material)	12	100%
Animation (8 types + stop)	9	100%
Formation arrangement (grid, circle, line)	8	100%
Structure generation (tower, castle, forest)	11	85%
Audio wiring (source → target mapping)	7	100%
Lighting (6 named themes + custom)	7	100%
Scene management (clear, undo, redo)	5	100%
Movement (direction + distance)	3	100%
VFX, atmosphere, removal, other	51	88%
Total (14 categories)	**143**	**91%**

Table 3. Voice intent parsing: 143 automated tests across 14 semantic action categories. Mean parse latency < 1 ms (synchronous regex). Low-confidence requests fall through to a cloud LLM.

This architecture matters on edge devices because it prioritizes low-friction local edits for common commands while preserving a richer fallback path for open-ended requests. Compared with fully cloud-routed language interfaces such as LLMR [29], Portals uses local parsing to keep the common authoring path responsive and reserves remote model calls for cases where semantic ambiguity is genuinely high.

Complementing voice, no-code authoring interfaces expose the same scene operations through composer controls. Taken together, VFX tooling, voice interaction, and no-code authoring expand who can build 4D content, shorten iteration cycles, and create pathways for collecting real-world usage data across deployment settings.

7. Dynamic 4D Content: Human Avatars and Live Events

Social spatial computing depends fundamentally on *human presence*: users do not simply want to see Gaussian-splatted environments—they want to see other people inhabiting those environments.

We integrate the Generalizable Human Gaussians (GHG) paradigm [14] for on-device avatar rendering. Input is a single RGB image or a short video; GHG produces a feed-forward 3DGS human model at interactive latencies on mobile hardware. For live events, we extend this with our real-time motion-capture pipeline (developed by Justin Kasowski), which streams skeletal pose estimates from iPhone TrueDepth sensors and retargets them onto Gaussian avatar representations at interactive end-to-end latency.

The combination of world-anchored Gaussian environments (4D-GS [9]), feed-forward human avatars (GHG [14]), and real-time skeletal animation (DELIFFAS [13]-inspired) forms a complete *social 4D world*: a shared, physically lo-

cated, temporally evolving scene populated by photorealistic avatars.

7.1. Live Holograms and Spatial Media Recording

The same shared runtime substrate also supports live and recorded holographic media. Portals adopts a UV-multiplexed spatial video format inspired by Keijiro Takahashi's MetavidoVFX[1]: a single 1920×1080 video frame packs RGB color (right half), hue-encoded depth (top-left quarter), human stencil (bottom-left quarter), and a metadata barcode burnt into the stencil channel encoding camera pose, field of view, depth range, and projection parameters (12 floats per frame). Because the multiplexed frame is a standard video texture, it can be recorded, streamed, and replayed through commodity H.264/HEVC codecs and standard video players — no bespoke spatial-format infrastructure is required.

On playback, a GPU demux shader (∼2 ms) splits the frame back into color, depth, and stencil planes; the decoded metadata reconstructs camera pose; and the resulting textures are bound through the same `ARDepthSource`/`VFXARBinder` path used for live VFX. Because the binding layer abstracts the data source, all 370+ production VFX effects work identically on live AR input and on decoded spatial video, enabling a seamless record–edit–replay loop over holographic content. The same format naturally extends to additional encoded channels (semantic labels, Gaussian splat parameters, VFX state), suggesting a path toward richer spatial-media containers that remain compatible with standard video distribution.

This design matters because recording and livestreaming are often where spatial demos break in deployment: Portals' approach keeps spatial formats compact enough to capture on-device, expressive enough to preserve depth-driven effects, and fully compatible with the live rendering path.

8. Discussion: Open Problems

8.1. Reconstruction Quality vs. Capture Burden

Even with ReconFusion-style priors [15], casual phone video introduces blur, illumination changes, and dynamic objects (moving people, cars) that degrade reconstruction quality. K-Planes [7] handles dynamic objects by factoring space-time planes, but training time (minutes) remains too long for in-situ AR capture. Recent single-video systems such as Shape of Motion [11] and MoSca [10] improve robustness from casual video, but the community still needs real-time or near-real-time 4D reconstruction from monocular capture at the frame rates required for live AR deployment.

[1] `https://github.com/keijiro/MetavideoVFX`

8.2. Scene Composition and Editability

Once reconstructed, users need to *edit* 4D scenes: remove unwanted objects, change lighting, insert synthetic content. Current 3DGS representations are notoriously hard to edit due to the lack of explicit semantic structure. Feature-field extensions [21] begin to address this, but compositing a Gaussian avatar into a Gaussian background without seams remains an open challenge.

8.3. Privacy in Persistent World Models

Persisting 3D scans at scale raises privacy concerns: bystanders captured inadvertently, interiors scanned without consent, geospatial databases aggregating personal data. Portals enforces client-side ephemeral buffers and uploads only user-selected keyframes. A principled privacy-by-design framework for persistent 4D world models remains open.

8.4. Toward Production Benchmarks for 4D World Models

Existing benchmarks (NeRF Synthetic, Tanks and Temples, MipNeRF-360, DiVa-360 [26]) evaluate reconstruction quality in controlled settings but omit metrics critical for production deployment. We propose four benchmark axes, of which we evaluate the first (Table 2) and leave axes 2–4 as community challenges: (1) *mobile rendering efficiency* — median and p95 frame times on commodity mobile GPUs as a function of scene complexity (Table 2 provides an initial baseline); (2) *geospatial alignment accuracy* — registration error (cm) after session re-entry across GPS conditions; (3) *temporal persistence fidelity* — anchor recovery rate, appearance drift, edit survival, and metadata-only versus full-payload reload cost across return visits; and (4) *creator editability* — semantic-action success rate, correction rate, and edit-to-preview latency for voice and no-code authoring surfaces, capturing whether a world model remains editable after leaving the training pipeline.

8.5. Extensible Spatial Intelligence Layers

A longer-term research opportunity is to treat persistent spatial scenes as an extensible substrate for additional intelligence layers rather than as fixed outputs of a single reconstruction or generation model. In Portals, semantics, body/hand/face signals, multimodal commands, live media transforms, and personalized scene preferences can all attach above the same persisted world state and shared runtime path. This lets enrichment accumulate over time: users can revisit the same scene through different interfaces or preferences, and researchers can compare new perception or generation modules against a common spatial-state representation rather than rebuilding the full stack for each experiment.
The Khronos addition of Gaussian Splats to glTF[2] which Por-

[2] Khronos `KHR_gaussian_splatting` release candidate, Feb. 2026; ratification expected Q2 2026.

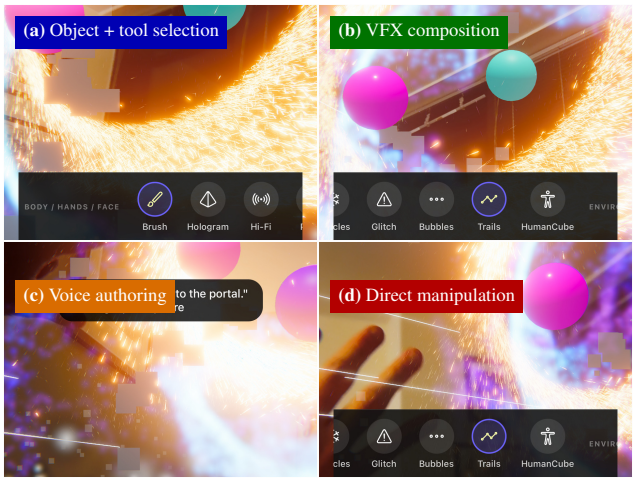

Figure 3. Real in-app creator-authoring captures. **(a)** Brush, Hologram, and Hi-Fi tool selection with placed 3D objects and active VFX particles. **(b)** VFX composition toolbar (Glitch, Bubbles, Trails, HumanCube) over a live AR scene. **(c)** Voice command "add a pink sphere to the portal" parsed and executed locally. **(d)** Direct manipulation of scene objects with active VFX and depth-driven effects. All four interfaces expose the same persistent scene graph through the composition bridge.

tals has adopted, provides the interoperability layer needed for cross-platform benchmark evaluation. We encourage the community to develop shared evaluation protocols around these axes.

9. Conclusion

We have presented Portals as a deployed 4D world-model system for the edge-device era. The paper's core result is not a single isolated representation, but a deployment stack that proves three coupled requirements can be satisfied together: real-time runtime performance on commodity mobile hardware, persistent scene state across revisits, and creator-facing world editing that bridges reconstruction and generation. Deployment experience spanning clinical AR [23], large live experiences, and consumer spatial publishing suggests that these requirements can compose into a viable deployment surface for spatial computing.

More broadly, we argue that this line of research matters because the next wave of world-model deployment will happen at the edge: on phones today, and increasingly on smart glasses and mixed-reality headsets. As spatial computing becomes a primary interaction paradigm, the bottleneck shifts from scene quality alone to deployable systems architecture: how dynamic scenes are rendered under tight thermal and memory budgets, how world state survives return visits, and how creators edit those worlds without expert-only tools. The same layered architecture also leaves room for spatial intelligence modules that enrich persistent scenes with new semantics, multimodal interaction modes, and personalized behaviors over time.

The next chapter — real-time 4D reconstruction from casual monocular video, semantically-editable Gaussian scenes, and privacy-preserving persistent world maps — requires continued collaboration between academic research and industry deployment, supported by the production benchmarks we propose.

Acknowledgements. This work was supported in part by the NIH/NCI Cancer Center Support Grant P30 CA008748 (COVIDvis component). The authors thank Dr. Omer Aras (Memorial Sloan Kettering Cancer Center) and Dr. Haluk B. Sayman (Istanbul University) for clinical collaboration. We also thank the open-source 3DGS community for tooling, the Apple Vision Pro developer program for early hardware access, and the H3M engineering team for production infrastructure.

References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of ECCV*, 2020.

[2] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan. Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields. In *Proceedings of ICCV*, 2021.

[3] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of CVPR*, 2022.

[4] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman. Zip-NeRF: Anti-aliased grid-based neural radiance fields. In *Proceedings of ICCV*, 2023.

[5] D. Verbin, P. Hedman, B. Mildenhall, T. Zickler, J. T. Barron, and P. P. Srinivasan. Ref-NeRF: Structured view-dependent appearance for neural radiance fields. In *Proceedings of CVPR*, 2022.

[6] A. Yu, S. Fridovich-Keil, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa. Plenoxels: Radiance fields without neural networks. In *Proceedings of CVPR*, 2022.

[7] S. Fridovich-Keil, G. Meanti, F. Warburg, B. Recht, and A. Kanazawa. K-Planes: Explicit radiance fields in space, time, and appearance. In *Proceedings of CVPR*, 2023.

[8] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (SIGGRAPH)*, 2023.

[9] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang. 4D Gaussian splatting

for real-time dynamic scene rendering. In *Proceedings of CVPR*, 2024.

[10] J. Lei, Y. Wang, C. Ye, J. Gu, A. Kanazawa, C. Daniilidis, P. Moulon, and G. Pavlakos. MoSca: Dynamic Gaussian fusion from casual videos via 4D motion scaffolds. *arXiv:2405.17421*, 2024.

[11] Q. Wang et al. Shape of Motion: 4D reconstruction from a single video. In *Proceedings of ICCV*, 2025.

[12] Y. Kwon, D. Kim, J. Ceylan, and H. Li. Neural human performer: Learning generalizable radiance fields for human performance rendering. In *Proceedings of NeurIPS*, 2021.

[13] Y. Kwon, L. Liu, H. Fuchs, M. Habermann, and C. Theobalt. DELIFFAS: Deformable light fields for fast avatar synthesis. In *Proceedings of NeurIPS*, 2023.

[14] Y. Kwon, B. Fang, Y. Lu, H. Dong, C. Zhang, F. V. Carrasco, A. Mosella-Montoro, J. Xu, and F. De la Torre. Generalizable human Gaussians for sparse view synthesis. In *Proceedings of ECCV*, 2024.

[15] R. Wu, B. Mildenhall, P. Henzler, K. Park, R. Gao, D. Watson, P. P. Srinivasan, D. Verbin, J. T. Barron, B. Poole, and A. Holynski. ReconFusion: 3D reconstruction with diffusion priors. In *Proceedings of CVPR*, 2024.

[16] B. Roessle, N. Müller, L. Porzi, S. R. Bulò, P. Kontschieder, A. Dai, and M. Nießner. L3DG: Latent 3D Gaussian diffusion. In *SIGGRAPH Asia Conference Papers*, 2024.

[17] S. Rota Bulò, L. Porzi, and P. Kontschieder. Revising densification in Gaussian splatting. In *Proceedings of ECCV*, 2024.

[18] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud. DUSt3R: Geometric 3D vision made easy. In *Proceedings of CVPR*, 2024.

[19] Q. Wang et al. CUT3R: Continuous 3D perception model with persistent state. In *Proceedings of CVPR*, 2025.

[20] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger. Mip-splatting: Alias-free 3D Gaussian splatting. In *Proceedings of CVPR*, 2024.

[21] M. Qin, W. Li, J. Zhou, H. Wang, and H. Pfister. LangSplat: 3D language Gaussian splatting. In *Proceedings of CVPR*, 2024.

[22] T. Xu et al. LODGE: Level-of-detail large-scale Gaussian splatting with efficient rendering. *arXiv:2505.23158*, 2025.

[23] H. B. Sayman, K. N. Toplutas, J. Tunick, and O. Aras. ^{99m}Tc-Vitamin C SPECT/CT imaging in SARS-CoV-2 associated pneumonia. *Nuclear Medicine Review*, 25(2):127–128, 2022. DOI: 10.5603/NMR.a2022.0026.

[24] J. Lin, Z. Li, X. Tang, J. Liu, S. Liu, J. Liu, Y. Lu, X. Wu, S. Xu, Y. Yan, and S. Zhang. VastGaussian: Vast 3D Gaussians for large scene reconstruction. In *Proceedings of CVPR*, 2024.

[25] J. Luiten, G. Kopanas, B. Leibe, and D. Ramanan. Dynamic 3D Gaussians: Tracking by persistent dynamic view synthesis. In *Proceedings of 3DV*, 2024.

[26] A. Xing, R. Rai, S. Sridhar, et al. DiVa-360: The dynamic visual dataset for immersive neural fields. In *Proceedings of CVPR (Highlight)*, 2024.

[27] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *Proceedings of CVPR*, 2017.

[28] Z. Xu, S. Peng, H. Lin, G. He, J. Sun, Y. Shen, H. Bao, and X. Zhou. 4K4D: Real-time 4D view synthesis at 4K resolution. In *Proceedings of CVPR*, 2024.

[29] F. De La Torre, C. M. Fang, H. Huang, A. Banburski-Fahey, J. Amores Fernandez, and J. Lanier. LLMR: Real-time prompting of interactive worlds using large language models. In *Proceedings of ACM CHI (Honorable Mention)*, 2024.

[30] J. Kasowski, B. A. Johnson, R. Neydavood, A. Akkaraju, and M. Beyeler. A systematic review of extended reality (XR) for understanding and augmenting vision loss. *Journal of NeuroEngineering and Rehabilitation*, 19(1):1–18, 2022.